

GPT2 Model Architecture

David Yuan

December 31, 2025

1 ARCHITECTURE

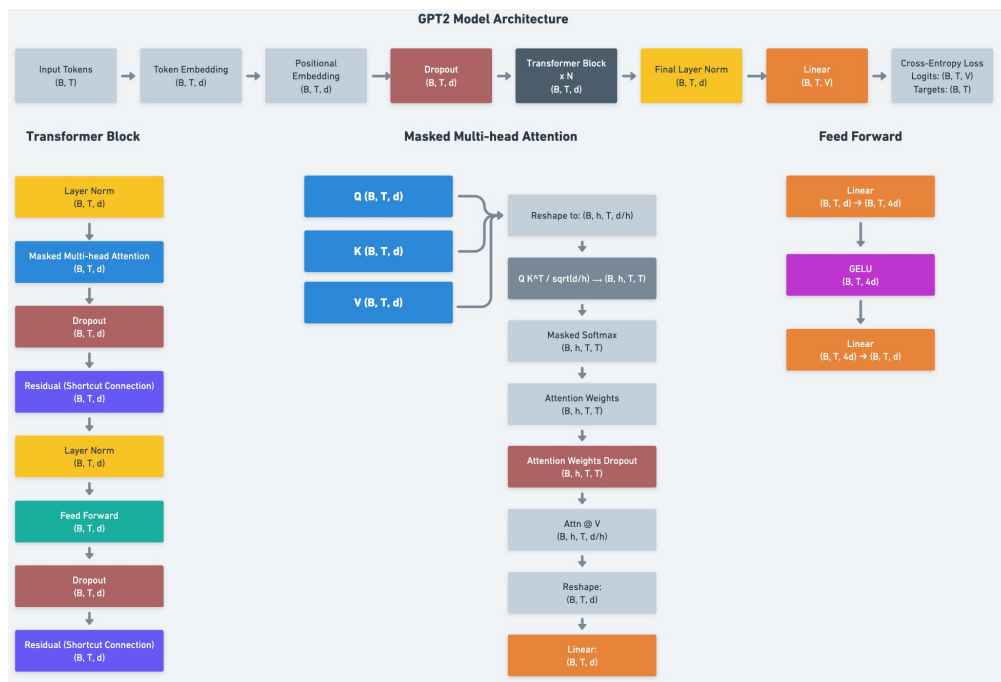


Figure 1.1: GPT2 model architecture

2 DETAILS

- B = batch size
- T = sequence length
- V = vocabulary size
- d = embedding dimension (e.g. 768)
- h = number of attention heads
- $d_{head} = d/h$

2.1 INPUT TOKENS

The raw text inputs are tokenized to tokens, each with an integer token ID (e.g. Byte Pair Encoding ; BPE). The input sample shape is: (B, T) .

2.2 TOKEN EMBEDDING

We start with a random token lookup table with shape: (V, d) . Then for each token we get:

$$token_{emb} = Embedding(V, d)(tokens)$$

The input: (B, T)

The output: (B, T, d)

2.3 POSITIONAL EMBEDDING

Lookup positional vectors for each token position.

$$pos_{emb} = Embedding(T_{max}, d)(positions)$$

Input: (T)

Output: (T, d)

2.4 TRANSFORMER BLOCK

The transformer block will repeat N times. Each block takes inputs (B, T, d) and returns (B, T, d) .

2.4.1 LAYER NORM

In the layer normalization, we generally will add two trainable parameters *scale* and *shift* that let LLM automatically adjusts during the training.

Input: (B, T, d)

Output: (B, T, d)

2.4.2 MASKED MULTI-HEAD ATTENTION

We initialize three matrices that are trainable W_q, W_k, W_v , each with shape (d, d) . Generally speaking, the input and output dimensions are the same.

$$Q = x @ W_q \rightarrow (B, T, d)$$

$$K = x @ W_k \rightarrow (B, T, d)$$

$$V = x @ W_v \rightarrow (B, T, d)$$

Then, split the last dimension into heads:

$$Q \rightarrow (B, T, h, d_{head})$$

$$K \rightarrow (B, T, h, d_{head})$$

$$V \rightarrow (B, T, h, d_{head})$$

Then rearranges all Q, K, V to:

$$(B, h, T, d_{head})$$

Attention scores are:

$$scores = Q @ K^T$$

where the shapes are:

- $Q \rightarrow (B, h, T, d_{head})$
- $K^T \rightarrow (B, h, d_{head}, T)$
- $scores \rightarrow (B, h, T, T)$

After the mask and softmax, the attention weights are still with shape: (B, h, T, T) .

Then, the context is:

$$context = attn @ V \rightarrow (B, h, T, d_{head})$$

Flatten head and reshape:

$$context \rightarrow (B, T, d)$$

2.4.3 FEED FORWARD

The feed forward is a MLP with:

- input: (B, T, d)
- hidden: $(B, T, 4d)$
- output: (B, T, d)

2.5 LANGUAGE MODEL HEAD

Project hidden states to vocabulary logits:

$$\text{logits} = \text{Linear}(d \rightarrow V)(x)$$

Input: (B, T, d)

Output: (B, T, V)

2.6 LOSS COMPUTATION

Targets:

- targets shape: (B, T)
- logits shape: (B, T, V)

Flatten:

- targets shape: $(B * T)$
- logits shape: $(B * T, V)$

Cross-entropy loss over all tokens:

$$\text{loss} = \text{mean}(-\log P(\text{correct token}))$$